

Boston Neighborhood Intelligence

Product Requirements & Build Plan — RAG the City, Track A “The Engine”

Saturday, August 22, 2026 · Build window 10:45 AM – 3:15 PM ET · 5-person team

Contents

1. Executive Summary	3
What we are building	3
The core technical thesis	3
Why this wins Track A	3
2. How We Are Scored	4
The anchors we must satisfy	4
Red-flag caps — these override good work	4
Two rules that shape how we work	4
3. What We Learned From The Real Data	5
Finding 1 — The crime data cannot answer “how much crime?” on its own	5
Finding 2 — Geography does not join, but 311 is a Rosetta Stone	5
Finding 3 — Aggregation is free, embedding is not	6
4. Datasets	7
The five datasets	7
The six reference documents	7
Scope decisions (locked)	8
5. Architecture	9
The six routes explained	9
Why retrieval is load-bearing (and not decoration)	10
The critical design split	10
6. Product Requirements	11
6.1 The Neighborhood Scorecard	11
6.2 Citation requirements	11
6.3 Abstention requirements	11
6.4 The honesty feature (a scoring goldmine)	11
6.5 Innovation plays	12

7. Data Ingestion Strategy	13
Tier 1 — DuckDB: 100% of rows, zero ingestion cost	13
Tier 2 — Chroma: deliberately tiny and synthesized	13
Why this answers the hardest judge question	13
Document parsing — parse once, commit the artifact	13
8. Frameworks and Tools	15
Already installed and verified	15
To add tomorrow (small, fast installs)	15
Retrieval techniques we implement	15
9. LLM Model Selection	16
Generation models	16
Embedding models	16
Recommended configuration	17
10. Work Division — 5 Teammates	18
Dependency map — who blocks whom	18
11. Timeline	19
Before the clock starts — 9:30 to 10:45, everyone, no code	19
The first fifteen minutes — 10:45 to 11:00, everyone	19
The build	19
The eval table we must commit	19
Cut list if behind	20
12. Risks	21
13. Definition of Done	21
14. Eligibility Gate — before 3:15 PM	21

1. Executive Summary

What we are building

A question-answering engine over Boston’s open data that serves **two personas**:

1. **The new resident** — *“Should I live in this neighborhood?”* Safety, city responsiveness, parks, food safety, housing cost.
2. **The city manager** — *“How is each neighborhood performing?”* SLA compliance, backlog, departmental comparison, service equity.

Both personas want **the same computation, framed differently**: the resident asks about one neighborhood, the manager ranks all of them. So we build **one Neighborhood Scorecard engine**, not two products.

The core technical thesis

Numbers come from SQL over 100% of rows. Retrieval’s job is to make that SQL correct.

We never ask a language model to read thousands of rows and eyeball a total — that is where naive RAG fabricates. Instead:

- **DuckDB** computes every number over the full dataset, deterministically.
- **Retrieval** supplies the model with what the columns *mean*, so the SQL it writes is right.
- **Lookup tables** (offense codes, value indexes) are *joined in SQL*, never retrieved.

Why this wins Track A

We are scored out of 16 on four criteria. Three are judged from the repo. This design targets the top anchor on each:

Criterion	How we hit it
RAG Quality & Grounding /4	Citations on every answer; explicit “the data does not say”; retrieval that is precise because it is schema-grounded
Track Excellence /4 (Track A)	Hybrid search + multi-source orchestration + agentic routing + a real eval pipeline with committed numbers
Innovation /4	Two genuinely novel findings (see §3), a derived geography crosswalk, and summary-card chunking for tabular data
Presentation /4 (live)	A demo built around a correctness story, not a feature tour

2. How We Are Scored

Source: the official rubric shipped in the starter repo at `.claude/skills/rag-city-judge/references/`. Everyone on the team should read this section.

16 points total. Four criteria, each an integer 1–4. No halves. Judges score immediately after each demo, against written anchors, not against other teams.

The anchors we must satisfy

RAG Quality & Grounding — the 4 anchor, verbatim:

“Every answer grounded in specific data with clear citations. Retrieval is precise and comprehensive — and it says ‘I don’t know’ instead of guessing.”

Three separate requirements: **citations, retrieval quality, abstention.**

Track A Excellence — the 4 anchor, verbatim:

“Sophisticated architecture — hybrid search, multi-source orchestration, agentic retrieval, or a real evaluation pipeline. Measurably better than naive RAG, and they can show the numbers.”

The judges’ review lens is blunt about the final clause: **“No numbers, no 4.”** An eval script with no committed output does not count.

Red-flag caps — these override good work

Red flag	Effect
Specifics that cannot be traced to a dataset	Caps RAG Quality at 1
Refusing off-script queries, steering back to rehearsed prompts	Caps RAG Quality at 2
A recording passed off as a live demo	Caps Presentation, undermines RAG Quality
Private / scraped / PII / credentialed data	Eligibility flag to organizers
Code not written today	Eligibility flag to organizers

The off-script cap is the one that quietly kills teams. If a judge types an unrehearsed question and our system deflects, we are capped at 2 on our most important criterion. **The system must attempt every question about our data.** Person E owns this.

Two rules that shape how we work

1. Starter plumbing earns no merit. From the judges’ provenance rules: if a repo is starter-derived, *“unmodified starter plumbing ... earns no team merit.”* Judges detect it by grepping for "Fort Point Files", "Millbrook", "granite3.1-dense", lab0, and by checking git ancestry.

-> **We start a brand-new empty repository. We write everything ourselves.**

2. Fresh code only. All code must be created inside the build window. Old commits are an eligibility flag.

-> **Nobody writes solution code before 10:45 AM.** Tonight is environment setup only.

3. What We Learned From The Real Data

We inspected the live datasets before planning. Three findings drive the whole architecture.

Finding 1 — The crime data cannot answer “how much crime?” on its own

Verified against the live Crime Incident Reports CSV (111 sampled rows):

- OFFENSE_CODE_GROUP — **100% empty**
- UCR_PART — **100% empty** (this is the violent-vs-property classification)
- Only OFFENSE_CODE (numeric) and free-text OFFENSE_DESCRIPTION are populated

And the descriptions include:

```
SICK ASSIST          <- not a crime
INVESTIGATE PERSON  <- not a crime
SERVICE TO OTHER AGENCY <- not a crime
INVESTIGATE PROPERTY <- not a crime
ASSAULT – AGGRAVATED  <- violent crime
MURDER, NON-NEGLIGENT MANSLAUGHTER
```

Consequence: a naive `SELECT count(*) ... GROUP BY district` counts sick assists as crimes and materially overstates every neighborhood’s crime rate. For a “should I live here?” product, that is not a rounding error — it is the entire answer being wrong.

The fix requires an external document: `rmsoffensecodes.xlsx` from the portal, parsed and joined as a dimension table. **This is why our document layer is a precondition for a correct number, not decoration.** It is also Innovation play #1: we can show naive-vs-ours crime counts side by side and say “*naive counts SICK ASSIST as a crime; we don’t.*”

Finding 2 — Geography does not join, but 311 is a Rosetta Stone

Every dataset names places differently:

Dataset	Column	Example value
311	neighborhood	"Allston / Brighton" — and separate "Allston", "Brighton" rows
Crime	DISTRICT	"B2" (police district code)
Open Space	DISTRICT	"Allston-Brighton" (hyphenated)
Food Inspections	zip, address	02135
Property Assessment	ZIPCODE	02135

311 even contradicts *itself*: "South Boston / South Boston Waterfront" (8,275 rows) coexists with "South Boston" (845); likewise "Greater Mattapan" / "Mattapan". There is a meaningless "Boston" bucket (4,708 rows) and 254 nulls.

But 311 carries neighborhood, police_district, location_zipcode, ward, precinct, city_council_district, latitude and longitude on every single row. So we do not hardcode a gazetteer — we **derive** the crosswalk from 311 by modal co-occurrence. Verified working; all 12 police districts map, and we get a confidence score for free:

District	Neighborhood	Purity	District	Neighborhood	Purity
A7	East Boston	100.0%	E18	Hyde Park	78.9%
A15	Charlestown	100.0%	A1	Downtown / Financial	72.9%
C11	Dorchester	99.1%	B2	Roxbury	65.9%
E13	Jamaica Plain	91.7%	E5	Roslindale	61.4%
D14	Allston / Brighton	86.4%	B3	Greater Mattapan	59.6%
C6	South Boston / Waterfront	83.2%	D4	South End	47.7%

The purity column is a feature. Where a district spans several neighborhoods (D4 = 47.7%), the answer says so: *“derived from police district B2, which is 66% Roxbury by 311 volume — treat as approximate.”* Honest imprecision is exactly what the RAG Quality 4-anchor rewards.

The same derivation gives zipcode → neighborhood, bridging Food Inspections and Property Assessment, and a normalisation map that collapses 311’s self-inconsistency.

Finding 3 — Aggregation is free, embedding is not

Measured on our own machine:

```
SELECT type, count(*) AS n FROM read_csv_auto('311-service-requests.csv')
GROUP BY type ORDER BY n DESC LIMIT 5;
-- 78,143 rows scanned, exact counts, 0.44 seconds, zero import step
```

Meanwhile the Food Inspections file is **896,379 rows**. Embedding that many texts on a laptop would consume the entire build window. This asymmetry dictates our two-tier ingestion (§7).

4. Datasets

Requirement: minimum two, from Analyze Boston (data.boston.gov) or approved supplementary sources. We use **five datasets plus six reference documents** — which also earns the Innovation 3 anchor, “novel combination of data sources.”

The five datasets

#	Dataset	Size	Serves	Key columns
1	311 Service Requests	78,143 rows, 30 cols	Responsiveness, complaint themes, and the geography hub	on_time, open_dt, closed_dt, department, neighborhood, police_district, location_zipcode
2	Crime Incident Reports (2023–present)	multi-year	Safety	OFFENSE_CODE, OFFENSE_DESCRIPTION, DISTRICT, OCCURRED_ON_DATE, Lat, Long
3	Food Establishment Inspections	896,379 rows, 26 cols	Food safety	violation, viol_level, result, zip, property_id, legalowner
4	Open Space	small	Parks / green space	SITE_NAME, DISTRICT, ACRES, ZipCode, TypeLong
5	Property Assessment	~180k parcels	Housing cost, equity metric	ZIPCODE, assessed value, occupancy code

The six reference documents

These are what make the numbers correct. All small (a few pages each).

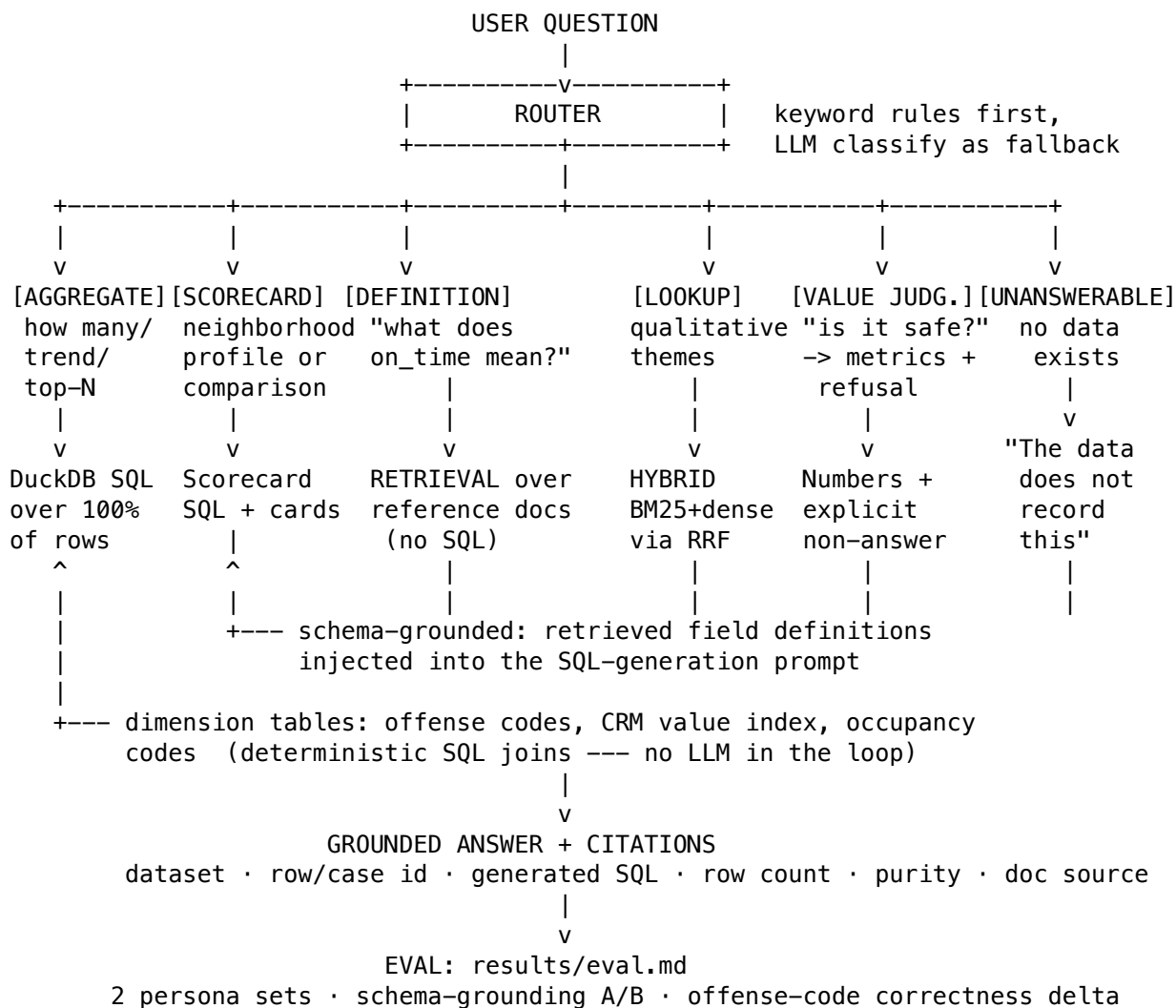
Document	Format	Destination	Priority
rmsoffensecodes.xlsx	XLSX	DuckDB dim table	Mandatory — see Finding 1
datadictionary-crmvaluecodeindex.pdf	PDF	DuckDB dim table	High — decodes 311 reason/type/queue
311-service-requests-data-dictionary-new-system.pdf	PDF	Vector store	High — grounds SQL generation
rmocrimeincidentfieldindex.xlsx	XLSX	Vector store	Medium
propertyoccupancycodes.pdf	PDF	DuckDB dim table	Medium — residential vs commercial for equity
property-assessment-fy2026-data-key.pdf	PDF	Vector store	Medium

Note: Food Establishment Inspections has **no documentation at all** on the portal. That is itself a finding, and it strengthens our abstention story — the data doesn’t say, and there isn’t even a dictionary to consult.

Scope decisions (locked)

- **Tourism: dropped.** No Analyze Boston dataset covers attractions. Recorded as a known limitation in the README. Honest scoping is rewarded; over-claiming is penalised.
- **Restaurants: reframed** to food safety only. The data gives hygiene, never quality or cuisine. We never recommend a restaurant.
- **We never rank “best neighborhood.”** See §6.4.

5. Architecture



The six routes explained

Route	Trigger	Engine	Example
AGGREGATE	“how many”, “average”, “trend”, “most”, “per year”	DuckDB SQL over 100% of rows	“How many rodent complaints in Dorchester last year?”
SCORECARD	a neighborhood name + a profile/compare intent	Scorecard SQL + summary cards	“How does Jamaica Plain look?”
DEFINITION	“what does X mean”, “how is X calculated”	Retrieval over reference docs, no SQL	“What does on-time mean for a 311 case?”
LOOKUP	qualitative / thematic	Hybrid BM25 + dense via RRF	“What do East Boston residents complain about?”
VALUE JUDGMENT	“is it safe”, “is it good”, “best”	Metrics + explicit refusal to rank	“Is Roxbury safe?”
UNANSWERABLE	low retrieval confidence, or no supporting data	Abstain	“Why was this licence suspended?”

Why retrieval is load-bearing (and not decoration)

A fair judge will ask: “if all your numbers come from SQL, where is the RAG?” Our answer has four legs, **two of which are preconditions for a correct number**:

#	Role	Mechanism	Remove it and...
1	Taxonomy resolution	Code tables joined in SQL	“Violent crime” becomes uncomputable; counts are wrong
2	Schema-grounded SQL	Retrieved field definitions injected into the SQL prompt	The model guesses what on_time / UCR_PART mean and writes wrong SQL. A/B measurable.
3	Definitional Q&A	Pure retrieval	Unanswerable from the CSV at all
4	Qualitative themes	Retrieval over free-text columns	No thematic questions work

The demo line: “We don’t ask the model to read the data — SQL does that, over 100% of rows. We use retrieval to teach the model what the columns mean, so the SQL it writes is right. Turn the retrieval layer off and our SQL accuracy drops from X% to Y%.”

That is a **correctness** argument, and it is measurable — which is exactly what the Track A 4-anchor demands.

The critical design split

Document type	Destination	Why
Code / lookup tables	DuckDB dimension tables	Deterministic joins. No LLM. Makes counts correct.
Prose field definitions	Vector store	Grounds SQL generation; answers definitional questions
Free-text CSV columns	Vector store	Qualitative themes

Codes are joined, never retrieved. This is the deterministic guarantee: we do not trust the model to remember that offense code 3115 is not a violent crime — we join a table.

6. Product Requirements

6.1 The Neighborhood Scorecard

Per canonical neighborhood, computed in SQL:

Dimension	Source	Metrics
Safety	Crime + offense-code dim	violent / property / non-crime split, trend, shootings (district-level, purity-labelled)
Responsiveness	311	median days to close, on_time %, open backlog, by department
Livability friction	311	top complaint types (rodents, potholes, trash)
Food safety	Food Inspections	% inspections with critical violations
Green space	Open Space	park acres, site count
Housing	Property Assessment	median assessed value

6.2 Citation requirements

Every answer carries: source dataset, row or case ID (or the generated SQL), row count, and where applicable the crosswalk purity and reference-document source. The rules are explicit: *“A system that can’t show its sources scores a 1 on RAG Quality. Citations are not a bonus — they’re the point.”*

For SQL answers, **print the generated query in the response**. The query *is* the citation, and it is highly persuasive to a judge.

6.3 Abstention requirements

Two gates:

1. **Pre-model gate** — if the fused top retrieval score is below threshold, abstain before calling the LLM at all.
2. **Prompt-level** — instruct the model that “the data does not record this” is a preferred answer over guessing.

Target: **zero fabrications** on the eval set.

6.4 The honesty feature (a scoring goldmine)

“Is Roxbury safe?” is a value judgment, not a data question. The system must answer:

“I can give you violent-crime counts and 311 response times for Roxbury with citations, but whether that means ‘safe’ is a judgment I won’t make for you. Here are the numbers.”

This does three things at once: hits the RAG Quality 4-anchor, avoids building a neighborhood-shaming machine, and is a strong live demo moment. Implemented as an explicit router branch, distinct from abstention.

6.5 Innovation plays

1. **The offense-code correctness delta.** Show naive-vs-ours crime counts for the same district. Concrete, novel, civically meaningful.
2. **The equity gap metric.** 311 median response time vs median assessed property value per neighborhood. If wealthier neighborhoods get faster service, that is a real finding — and exactly what persona 2 exists to discover. Joins on `location_zipcode <-> ZIPCODE`, so it has **no dependency on the geography crosswalk** and can start immediately.

7. Data Ingestion Strategy

Two tiers. This is the answer to “how much data do we ingest?”

Tier 1 — DuckDB: 100% of rows, zero ingestion cost

Point DuckDB at the CSV files directly with `read_csv_auto`. **There is no import step and no sampling.** Verified: 78,143 rows aggregated in 0.44s. All five datasets together are ~1.5M rows and will still answer in roughly 1–2 seconds.

Build one SQL **view per dataset** that adds a canonical neighborhood column via the derived crosswalk, plus **dimension tables** from the parsed reference documents. Every count, rate, trend and comparison runs live against full data. **We never approximate a number.**

Tier 2 — Chroma: deliberately tiny and synthesized

Do **not** bulk-embed raw rows. Embed three things:

Content	Approx. count	Purpose
Reference-doc chunks	~200	Schema-grounded SQL + definitional Q&A
Scorecard cards	~100	~16 neighborhoods × ~6 dimensions. Each is a natural-language paragraph <i>generated from the SQL aggregates</i> , carrying source dataset, row count, purity and generating SQL as metadata.
Sampled raw text	~3,000–4,000	311 case_title + closure_reason, food violdesc + comments. One row per chunk, metadata preserved.

Hard cap: 5,000 documents, enforced by a constant in code. Index builds in under two minutes.

Why this answers the hardest judge question

Judge probe: *“How did you chunk your largest CSV so retrieval actually works?”* — described in the judges’ own material as “the day’s hardest Track A problem.”

Our answer: *“We didn’t chunk it as text at all. Embeddings can’t count, so numbers go to SQL over 100% of rows. We aggregate first, then embed the narrative of the aggregate — our retrieval unit is a computed summary, not a row fragment.”*

That is a novel chunking strategy for tabular data, which the event page names as an explicit Track A bonus signal.

Document parsing — parse once, commit the artifact

Need	Tool	Rationale
Data-dictionary PDFs (table-shaped)	pdfplumber	Pure Python, tiny install, <code>extract_tables()</code> is excellent on ruled/aligned tables. Primary choice.
Prose pages / raw speed	PyMuPDF (fitz)	Very fast, lightweight, reliable. Fallback.
XLSX code tables	pandas.read_excel + <code>openpyxl</code>	Trivial
<i>Optional upgrade</i>	Docling	Named in the judges' Track A review lens; best table fidelity. But pulls ML layout models — install risk on a 4.5h clock. Only if someone has spare time.

Critical engineering call: these documents are static. Parse them early, write the results to `data/reference/*.csv`, and **commit those files**. The runtime pipeline reads committed CSVs, never PDFs. A parser failure at 2 PM then cannot break the demo, and the parsing code still lives in the repo as evidence of multi-source work.

8. Frameworks and Tools

All free, all open source, all running locally. **No cloud API keys** — the event page names “entirely local models” and “exclusively open-source tools” as explicit Track A bonus signals.

Already installed and verified

Tool	Version	Role
Python	3.13.3	Runtime
DuckDB	1.5.5	SQL over CSVs — the aggregation engine
ChromaDB	1.5.9	Dense vector store
rank_bm25	0.2.2	Sparse keyword retrieval
LangChain	1.3.16	Pipeline glue
LangGraph	1.2.11	Optional: router as an explicit state graph
pandas	3.0.5	Data wrangling
Ollama	0.32.14	Local model server
Streamlit	—	Optional demo surface (Presentation only)

To add tomorrow (small, fast installs)

Tool	Why
pdfplumber	Reference-document table extraction
pymupdf	PDF fallback
openpyxl	XLSX offense codes

Retrieval techniques we implement

Hybrid search. Dense (embedding) retrieval finds paraphrases; sparse (BM25) finds exact codes, IDs and raw numbers. We run both and merge with **Reciprocal Rank Fusion**:

$$\text{score}(d) = \sum_{r \in \text{retrievers}} \frac{1}{k + \text{rank}_r(d)} \quad k = 60$$

RRF ignores incompatible score scales and uses rank position only. It needs no training and is about fifteen lines of Python. This matters concretely: a figure like 4200000 in a bare CSV field is invisible to dense retrieval but trivial for BM25.

9. LLM Model Selection

Target hardware: M5 Mac, 24 GB unified memory. Budget roughly 16–18 GB for models, leaving headroom for the OS, DuckDB, Chroma and a browser. All sizes below are **verified against the live Ollama registry**.

Generation models

Model	Size	Best at	Verdict for us
qwen2.5-coder:7b	4.7 GB	Code and SQL generation	Recommended for the SQL path. Code-specialised models are substantially better at text-to-SQL than general models of the same size.
granite3.1-dense:8b	5.0 GB	Structured data, enterprise text	Recommended for synthesis + routing. Already pulled and tested. Strong at grounded summarisation.
qwen2.5-coder:14b	9.0 GB	Code and SQL, stronger	Upgrade path if 7B SQL accuracy disappoints. Slower per token but SQL outputs are short.
qwen2.5:14b	9.0 GB	Strong generalist	Good single-model option if we want simplicity over specialisation.
phi4:14b	9.1 GB	Reasoning	Strong reasoner; less proven on SQL.
qwen3:14b	9.3 GB	Newer generalist	Viable, but less battle-tested for our specific tasks.
mistral-nemo:12b	7.1 GB	Balanced generalist	Solid fallback, no particular edge for us.

Embedding models

This is where we get a free upgrade. **Ollama can serve embedding models**, which means better embeddings **without installing PyTorch** — avoiding a ~2.5 GB dependency and real install risk.

Model	Size	Notes
bge-m3	1.2 GB	Recommended. Multilingual — handles the Spanish, Haitian Creole and Chinese text that appears in real Boston 311 submissions. 1024-dim. Served by Ollama, no PyTorch.
nomic-embed-text	0.3 GB	Excellent English-only alternative, 8192-token context, very fast. Good if we want minimum footprint.
all-MiniLM-L6-v2	bundled	Chroma's default. 384-dim, English-only, weakest. Our fallback only.

Recommended configuration

Purpose	Model	Resident size
SQL generation	qwen2.5-coder:7b	4.7 GB
Answer synthesis + routing	granite3.1-dense:8b	5.0 GB
Embeddings	bge-m3	1.2 GB
Total		~11 GB of 24 GB

This leaves comfortable headroom, and Ollama can keep all three resident so there is no model-swap latency mid-demo.

Why two generation models rather than one: SQL generation and grounded prose synthesis are genuinely different skills. A 7B code model beats an 8B general model at SQL, and granite is better at citation-bearing prose. The cost is 4.7 GB of RAM we have to spare.

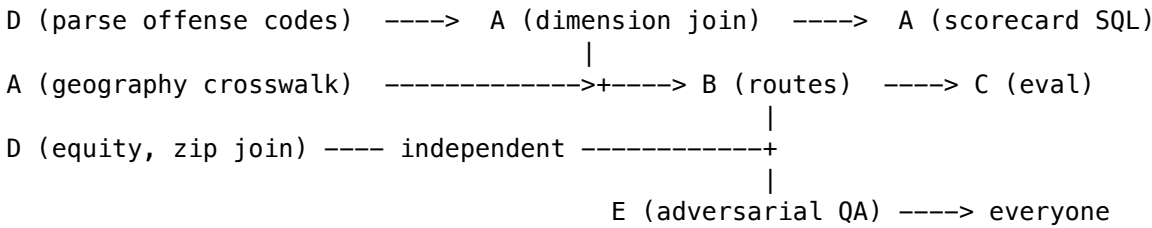
Pull all three tonight — model downloads are measured in gigabytes and conference WiFi will not save us.

Fallback: if local inference misbehaves, cloud APIs are permitted in both tracks. But running entirely locally earns Track A bonus points, so the cloud is a parachute, not a plan.

10. Work Division — 5 Teammates

Role	Owner	Mission	Primary deliverables
A — Data & SQL		The numbers are correct	Derived geography crosswalk; DuckDB views; dimension-table joins; scorecard SQL
B — Retrieval & Router		The right engine handles each question	Router; hybrid BM25+dense+RRF; schema-grounded SQL prompt; citation assembly
C — Eval & Docs		We can prove we are better	eval/questions.json; own naive baseline; results/eval.md; README
D — Documents & Equity		The reference layer and the novel finding	Parse offense codes + PDFs; commit artifacts; equity gap metric
E — Surface & Adversarial QA		Nothing breaks on stage	CLI, optional Streamlit; adversarial testing; demo video

Dependency map — who blocks whom



Two things are on the critical path. Everything else can proceed in parallel.

1. **D's offense-code parse** must land by 11:30 so A can build the crime dimension join. If it slips, fall back to hand-classifying the top 20 offense descriptions.
2. **A's geography crosswalk** unblocks every neighborhood-keyed query. It is ~15 minutes of SQL, not a hand-typed gazetteer. Ship a partial map covering the top 8 neighborhoods rather than a perfect one.

Deliberately independent: D's equity metric joins on ZIP, so it needs neither the crosswalk nor the offense codes. D can start it the moment the document parsing is handed off.

Person E is the insurance policy. The off-script cap (RAG Quality -> 2) is the most likely way we lose points. E's job from 13:00 onward is to break the system with questions nobody rehearsed, so that a judge cannot.

11. Timeline

Before the clock starts — 9:30 to 10:45, everyone, no code

Restart the Ollama server and confirm all three models respond. Download the five CSVs and six reference documents. Agree the canonical neighborhood list on paper. Write the ten demo questions (five per persona). **No solution code — this is an eligibility rule, not a guideline.**

The first fifteen minutes — 10:45 to 11:00, everyone

New empty repository, **first commit at or after 10:45**. README stub naming Track A, the five datasets, and a one-line pitch.

The build

Time	A – Data/SQL	B – Router	C – Eval	D – Docs/Equity	E – Surface/QA
11:00	Derived crosswalk + purity	Router skeleton, keyword rules	Question set scaffold	Parse offense codes + value index, commit CSV	Make targets, repo hygiene
11:30	DuckDB views + offense-code dim join ; scorecard SQL	Citation-carrying answer assembly	Own naive baseline	Parse remaining PDFs, commit	Thin CLI over the engine
12:15	SQL path wired; print generated SQL	Schema-grounded SQL prompt	Expected answers, ~20 questions	Equity join on ZIP	Streamlit, Presentation only
13:00	Scorecard cards from aggregates	Chroma + BM25 + RRF ; definition route	Run eval; schema-grounding A/B	Equity numbers, sanity check	Adversarial question list
13:45	Purity disclosure in answers	Tune top-k; fix retrieval misses	Commit the eval results file	Offense-code delta figure	Off-script run 1
14:15	Fix what E breaks	Fix what E breaks	README: architecture, eval table, limitations	Write up equity finding	Off-script run 2 — 10 unrehearsed questions
14:45	Freeze support	Freeze support	Final README + dataset list	—	Record 2-min live video, submit

15:15 — HARD CODE FREEZE. Repos locked, submissions in. Rehearse, do not debug.

The eval table we must commit

This is the artifact that converts a 3 into a 4. Illustrative shape — fill in real numbers:

Metric	Naive baseline	Ours	Delta
Overall accuracy	55%	85%	+30

Metric	Naive baseline	Ours	Delta
Counting questions correct	0 / 5	5 / 5	+5
Retrieval hit rate @5	0.61	0.92	+0.31
SQL accuracy <i>without</i> schema grounding	60%	—	—
SQL accuracy <i>with</i> schema grounding	—	90%	+30
Correct abstentions	0 / 4	4 / 4	+4
Fabrications	6	0	-6

Cut list if behind

Drop in this order: Streamlit surface -> lat/long crosswalk upgrade -> Open Space -> scorecard cards (fall back to raw-text retrieval) -> Food Inspections.

Never cut: the offense-code dimension table, citations, abstention, or the eval numbers.

12. Risks

Risk	Mitigation
<i>"Where's the RAG if everything is SQL?"</i>	§5 — four load-bearing roles, two of which are preconditions for correct numbers, with an A/B to prove it
D's parsing blocks A's dimension join	D parses offense codes first , hands off by 11:30. Fallback: hand-classify the top 20 offense descriptions.
A PDF parser fails	Parse-once-and-commit means failures surface at 11:30, not on stage. pdfplumber -> PyMuPDF -> manual transcription (docs are only a few pages).
Model writes invalid SQL	Validate before executing; retry once with the error appended; then fall back to retrieval. One view per prompt , never all five schemas at once.
District-level crime is coarse (D4 = 47.7% purity)	Disclose purity in answer text and README. Honest imprecision beats false precision — and the rubric rewards it.
Embedding too much, losing the window	Hard cap of 5,000 documents, enforced by a constant
Five datasets is ambitious	Ordered cut list above; two satisfies eligibility
E's UI causes scope creep	It is for Presentation only . Track A is not scored on UI — the rubric states a terminal app is never marked down for looking like a terminal. Freeze it at 13:00.
Judge asks an unrehearsed question	E's entire afternoon is spent preventing this
Disk space	Free space tonight. Models total ~11 GB.

13. Definition of Done

- ☐ **Offense codes:** every OFFENSE_CODE joins to the dim table; unmatched % reported. Hand-verify SICK ASSIST classifies as non-crime and ASSAULT – AGGRAVATED as violent.
- ☐ **Crosswalk:** every distinct raw geography value maps to a canonical neighborhood or an explicit UNKNOWN — no silent drops. Coverage % printed.
- ☐ **SQL path:** three scorecard numbers hand-checked against a direct DuckDB query. Exact match required.
- ☐ **Schema-grounding A/B:** SQL accuracy recorded both with and without retrieved field definitions, in results/eval.md.
- ☐ **Definitional route:** answers from reference docs, with the document cited.
- ☐ **Citations:** source metadata survives into every final answer string.
- ☐ **Abstention:** 4 unanswerable questions all decline. **Zero fabrications.**
- ☐ **Value judgment:** "is X safe?" returns metrics plus an explicit refusal to rank.
- ☐ **Off-script:** 10 unrehearsed questions — no crashes, every answer cited or declined.
- ☐ **Committed:** results/eval.md with naive-vs-ours numbers, broken out per persona.

14. Eligibility Gate — before 3:15 PM

- ☐ Public repository, **first commit after 10:45 AM**
- ☐ Five Analyze Boston datasets + reference documents listed in the README
- ☐ **2-minute demo video, recorded live** — a canned recording caps Presentation
- ☐ Track A declared (locked at 10:45, no switching)
- ☐ **No solution code written before 10:45 AM** — old commits are an eligibility flag

Rubric quotations are from `.claude/skills/rag-city-judge/references/rubric.md` and `track-a-review.md` in the official starter repo. Data findings (empty `OFFENSE_CODE_GROUP` and `UCR_PART`, the 311 geography columns, district purity figures, the 0.44s aggregation timing) are measured from the live Analyze Boston datasets on August 21, 2026. Model sizes are verified against the Ollama registry.